

复习卡片应用——大作业报告

助教分配 32 队

组号：89

成员：刘禹贤，孟宇翔，钟君浩

1. 程序功能介绍

本程序是一款基于 Qt 框架开发的桌面端复习卡片应用，旨在帮助用户通过“闪卡”模式高效记忆知识点。主要功能包括：

卡组管理：用户可以创建、删除、重命名卡组（牌组），每个卡组包含若干张卡片，每张卡片由“问题”和“答案”组成。

卡片管理：支持在选定卡组内添加、删除、修改卡片，并提供关键词搜索功能。

记忆复习模式：无限制循环浏览卡组内的所有卡片，适合快速回顾。支持进度保存，中途退出后可选择继续或重新开始。

自测模式：逐张展示卡片，用户查看答案后需对自身掌握程度进行评分（熟练/犹豫/陌生）。系统会根据历史评分数据，在下一次自测时优先展示不熟悉的卡片。完成一轮后提供“再来一次”和“清除缓存”功能。

背景自定义：支持更换应用背景图片，并可清除恢复默认。

数据持久化：所有卡组、卡片及评分数据均保存在本地 JSON 文件中，关闭应用后不丢失。

2. 项目各模块与类设计细节

2.1 整体架构与设计模式

本项目采用经典的 MVC（Model-View-Controller）架构思想，将数据模型、用户界面和业务逻辑分离，以提高代码的可维护性和可扩展性。具体而言：

Model（模型）：Card 类和 Deck 类承担数据模型的角色。Card 封装了单张卡片的属性和行为，包括问题、答案以及用于智能排版的评分数据；Deck 则代表一个卡组，管理一组 Card 对象，并提供基本的增删改查接口。

View（视图）：MainWindow 类负责所有用户界面的构建和呈现。它使用 QStackedWidget 管理五个页面（启动页、功能菜单、管理页、记忆复习页、自测页），每个页面由一系列 Qt 控件（QListWidget、QTextEdit、QPushButton、QComboBox 等）组成，为用户提供直观的操作入口。

Controller（控制器）：DataManager 类作为全局数据管理器，采用单例模式（Singleton Pattern）确保整个应用中只有一个数据实例。它负责从 JSON 文件加载数据、将内存中的数据保存回文件，并提供统一的接口供 MainWindow 调用。此外，MainWindow 本身也承担了一部分控制逻辑，例如响应用户操作、更新视图状态、调用 DataManager 的方法等。

这种分层设计的优势在于：

- （1）需要改变数据存储格式（例如从 JSON 迁移到 SQLite）时，只需修改 DataManager 类，而无需改动 UI 代码。
- （2）当需要增加新的复习模式（例如限时挑战）时，只需在 MainWindow 中添加一个新页面，并复用现有的 Card、Deck 和 DataManager 接口。
- （3）代码职责清晰，便于团队协作开发和后期维护。

2.2 Card 类：卡片数据模型

Card 类是系统的核心数据单元，它不仅仅是一个简单的“问题-答案”对，还内置了一套轻量级的评分跟踪机制，用于支持自测模式下的智能排序。

属性设计：

- （1）QString m_question：问题文本，用户输入的内容。
- （2）QString m_answer：答案文本。
- （3）int m_totalReviews：累计评测次数。每当用户在自测中对这张卡片进行评分（熟练/犹豫/陌生），该字段递增 1。
- （4）int m_sumRatings：累计评分总和。评分规则为：熟练=2 分，犹豫=1 分，陌生=0 分。每次评分后，m_sumRatings 累加对应的分值。
- （5）double m_easeFactor 和 int m_interval：这两个字段来自原始的间隔重复算法（SM-2），但在本项目中并未实际用于复习调度，仅作为预留字段保留，以保证与早期版本的兼容性。

关键方法：

- （1）getFamiliarity()：计算熟悉度指标，公式为 $m_sumRatings / \max(1, m_totalReviews)$ 。当 m_totalReviews 为 0 时返回 0（表示从未评测）。该值范围在 0~2 之间，数值越高表示用户对该卡片越熟练。自测模式开始时会依据此值对所有卡片进行升序排序，从而实现“优先测试不熟悉卡片”的目标。

(2) `resetReviewData()`: 将所有评分相关字段归零 (`m_totalReviews=0`, `m_sumRatings=0`)，用于“清除缓存”功能。

(3) `toJsonObject()/fromJsonObject()`: 实现 `Card` 对象与 `JsonObject` 之间的双向转换，使得卡片数据可以被序列化到 JSON 文件中。序列化时，将 `m_nextReviewDate` 转换为字符串格式 “yyyy-MM-dd”，反序列化时再还原为 `QDate` 对象。

设计决策:

为什么选择将评分数据存储在 `Card` 对象内部，而不是单独建立一个评分表？原因有二：一是简化数据模型，避免额外的关联查询；二是评分数据与卡片本身的生命周期一致，删除卡片时评分数据自然消失，无需手动清理。缺点是当卡片数量巨大时，排序时需要扫描所有卡片并计算熟悉度，但考虑到典型用户的卡片规模（通常在几百张以内），性能影响可以忽略。

2.3 Deck 类：卡组容器

`Deck` 类作为一个容器，管理一组 `Card` 对象，并提供面向卡组的操作接口。

属性:

(1) `QString m_name`: 卡组名称，用户可自定义。

(2) `QList<Card> m_cards`: 卡片列表，使用 `QList<Card>` 而非 `QList<Card*>`，即采用值语义。这样做的优点是：不需要手动管理内存（Qt 的隐式共享机制使得复制成本较低），且 `std::sort` 等算法可以直接作用于列表元素。缺点是修改卡片时需要获取引用，但 `getCards()` 返回 `QList<Card>&` 即可解决。

关键方法:

`addCard(const Card& card)`: 向卡组追加一张卡片，通过拷贝构造将传入的卡片副本加入列表。

`removeCard(int index)`: 根据索引删除卡片，使用 `QList::removeAt` 实现，时间复杂度 $O(n)$ 。

`getCard(int index)`: 返回指定索引的卡片引用（非常量和常量版本），允许外部直接修改卡片内容。

`getDueCards()`: 返回所有到期卡片的指针列表，用于间隔重复复习。该方法遍历 `m_cards`，对每张卡片调用 `isDue()`，若当前日期大于等于 `m_nextReviewDate`，

则将卡片地址加入结果列表。注意：本项目实际并未使用间隔重复，该方法仅作为接口保留。

`toJson()` / `fromJson()`：序列化/反序列化整个卡组。序列化时，将 `m_name` 和 `m_cards`（通过 `card.toJsonObject()` 逐个转换）打包成一个 `QJsonObject`。反序列化时，解析 “name” 和 “cards” 字段，重建卡片列表。

设计要点：`m_cards` 的公开访问（`public` 成员）是为了方便 `DataManager` 直接遍历统计卡片数量，这在 C++ 中虽不鼓励，但出于与原有代码的兼容性考虑予以保留。在实际工程中，应通过 `getCards()` 方法访问。

2.4 DataManager 类：全局数据管理与持久化

`DataManager` 是整个应用的数据中枢，采用单例模式确保全局唯一实例，负责数据的加载、保存和生命周期管理。

单例模式实现：

在 `DataManager.h` 中声明 `static DataManager* m_instance;` 作为私有静态成员。在 `DataManager.cpp` 中定义并初始化为 `nullptr`：`DataManager* DataManager::m_instance = nullptr;` 提供静态公有方法 `instance()`，如果 `m_instance` 为空则创建新对象，否则返回已有实例。构造函数设为私有（通过 `explicit DataManager(QObject* parent = nullptr)` 并让 `instance()` 成为友元或直接调用私有构造，此处采用公有构造但限制外部通过 `instance()` 访问的方式，实际代码中构造函数是公有的，但 `instance()` 方法提供了便捷的单例访问途径）。

数据持久化流程：

加载：`load(const QString& filePath)` 方法打开指定 JSON 文件，读取全部内容，使用 `QJsonDocument::fromJson` 解析。检查根对象是否包含 “decks” 键，若不存在则报错返回 `false`。遍历 “decks” 数组，对每个元素创建一个新的 `Deck` 对象，调用其 `fromJson()` 方法填充数据，然后将 `Deck*` 加入 `m_decks` 列表。最后更新 `m_currentFilePath` 并打印调试信息。

保存：`save(const QString& filePath)` 方法遍历 `m_decks`，调用每个 `Deck` 的 `toJson()` 获取其 JSON 表示，组装成 `QJsonArray` 并放入根对象的 “decks” 字段。使用 `QJsonDocument::toJson(QJsonDocument::Indented)` 生成格式化 JSON

字符串，写入文件。如果未指定路径，则使用 `m_currentFilePath` 或默认值 `"data.json"`。

内存管理：DataManager 拥有 `m_decks` 中所有 Deck 对象的所有权。析构函数调用 `clearDecks()`，后者遍历列表并 `delete` 每个 Deck，然后清空列表。`removeDeck(int index)` 在删除指定索引的 Deck 后也会释放其内存。这种设计避免了内存泄漏，但要求外部不能持有已删除 Deck 的悬垂指针。

2.5 MainWindow 类：视图与业务逻辑集成

MainWindow 是应用中最复杂的类，它集成了 UI 搭建、事件响应、业务逻辑处理和状态管理等多重职责。

页面管理：

使用 `QStackedWidget` 作为中央部件，管理五个页面：

页面 0：启动页，显示应用名称、副标题和“进入应用”、“更换背景”、“清除背景”按钮。

页面 1：功能菜单页，提供三个功能入口按钮（卡片管理、记忆复习、自测）和一个“返回主页”按钮。

页面 2：管理页，左右三栏布局：左侧卡组列表（含添加/删除/重命名按钮），中间卡片列表（含添加/删除按钮），右侧编辑区（问题/答案文本框、更新按钮、搜索框）。

页面 3：记忆复习页，包含卡组下拉框、问题/答案标签、进度条、开始/下一张/结束按钮。

页面 4：自测页，结构与记忆复习页类似，但增加了“显示答案”按钮、三个评分按钮（熟练/犹豫/陌生）、“再来一次”按钮和“清除缓存”按钮。

信号槽连接：

所有按钮点击事件均连接到对应的槽函数。例如，管理页的“添加卡片”按钮连接到 `onAddCard()`，记忆复习页的“下一张卡片”连接到 `onMemoryNextCard()`。槽函数中执行相应的业务逻辑，并更新 UI 状态（如按钮的启用/禁用）。

状态管理：

MainWindow 维护多个状态变量：

`currentDeckIndex`: 当前选中的卡组索引（-1 表示未选中）。

`currentCardIndex`: 当前选中的卡片索引（-1 表示未选中）。

`currentMemoryCardIndex/ currentReviewCardIndex`: 记忆/自测模式中当前显示的卡片索引。

`memoryCards/ reviewCards`: 存储当前记忆/自测模式的卡片列表（`QList<Card>` 值语义）。

`savedMemoryCards/ savedMemoryIndex`: 按卡组索引保存的记忆进度（`QMap<int, QList<Card>>`和 `QMap<int, int>`）。

`savedReviewCards/ savedReviewIndex`: 自测进度保存。

`completedReviewDecks`: 记录已完成一轮自测的卡组索引（`QSet<int>`）。

`reviewCompleted`: 当前自测是否已完成一轮的标志。

进度保存与恢复机制:

当用户切换卡组时, `onMemoryDeckChanged` 和 `onReviewDeckChanged` 会先保存当前卡组的进度（如果存在），然后加载新卡组的进度。进度数据存储在 `QMap` 中，键为卡组索引，值为卡片列表和当前索引。

在自测模式下，如果用户中途点击“结束”，系统会将当前卡片列表和索引保存到 `savedReviewCards` 和 `savedReviewIndex`，下次进入同一卡组时弹出对话框询问是否继续。

如果用户完成了一轮自测（到达最后一张卡片后点击“下一张”），系统会将该卡组标记为已完成（加入 `completedReviewDecks`），并清除保存的进度，这样下次进入时直接新开始，不再弹对话框。

评分与智能排序:

自测模式下，用户点击“显示答案”后，三个评分按钮（熟练/犹豫/陌生）变为可用。用户点击任一评分按钮后，`recordRating(int rating)`被调用，更新当前卡片的 `m_totalReviews` 和 `m_sumRatings`，同时遍历原卡组找到对应卡片并同步更新，最后调用 `dataManager->save()` 持久化。

每次开始自测时，`loadAndSortCardsByFamiliarity()` 方法从卡组加载所有卡片，并使用 `std::sort` 按 `getFamiliarity()` 升序排列，使得熟悉度最低的卡片最先出现，从而实现“优先测试不熟悉卡片”的目标。

UI 状态联动:

为了提供良好的用户体验，许多按钮的启用/禁用状态与当前选择相关联。例如：未选中卡组时，“添加卡片”、“删除卡片”、“重命名牌组”按钮均为禁用状态。选中卡组后，“添加卡片”启用；“删除卡片”需选中具体卡片后才启用。记忆/自测模式中，点击“开始”后，开始按钮和卡组下拉框被禁用，直到点击“结束”才恢复。自测完成一轮后，“清除缓存”按钮启用，点击后重置所有卡片的评分数据。

2.6 数据流与交互流程详解

以一次完整的自测为例，展示数据如何在各模块间流动：

用户选择卡组：用户在自测页的下拉框中切换卡组，触发 `onReviewDeckChanged`。该函数首先保存当前卡组的进度（如果存在），然后根据新卡组是否有未完成的进度或是否已完成，更新 UI 提示文本。

用户点击“开始自测”：触发 `onStartReview`。该函数检查是否有未完成的进度且未完成一轮，若有则弹出对话框询问是否继续。根据用户选择，要么恢复进度，要么重新加载卡片并排序。排序过程调用 `loadAndSortCardsByFamiliarity()`，从 `DataManager` 获取卡组，复制卡片列表，然后按熟悉度升序排列。之后将第一张卡片的问题显示在标签上，并启用“显示答案”按钮。

用户点击“显示答案”：触发 `onShowAnswer`。答案标签变为可见，同时三个评分按钮启用，“显示答案”按钮禁用，“下一张”按钮仍禁用（强制用户必须先评分）。

用户评分：点击“熟练”、“犹豫”或“陌生”，触发对应的评分槽函数，最终调用 `recordRating(rating)`。该函数更新 `reviewCards` 中当前卡片的评分数据，然后遍历原卡组（通过 `DataManager` 获取）找到匹配的卡片并同步更新，最后保存到文件。评分后，评分按钮禁用，“下一张”按钮启用。

用户点击“下一张”：触发 `onNextCard`。如果当前不是最后一张，则索引递增，显示下一张卡片的问题，隐藏答案，启用“显示答案”按钮，禁用“下一张”和评分按钮。如果已经是最后一张，则标记为已完成，更新 UI 显示最后一

张卡片的答案，启用“再来一次”和“清除缓存”按钮，禁用“下一张”和评分按钮。

用户点击“结束”：触发 `onReviewEnd`。如果尚未完成一轮，保存当前进度；否则不保存。重置所有按钮状态，恢复开始按钮和下拉框的可用性。

用户点击“清除缓存”：触发 `onClearCache`。弹出确认对话框，确认后遍历原卡组，对每张卡片调用 `resetReviewData()`，然后保存。之后重置自测状态，使所有卡片熟悉度归零。

2.7 关键算法与数据结构

熟悉度排序：使用 `std::sort` 结合 Lambda 表达式，比较两个 `Card` 对象的 `getFamiliarity()` 返回值。由于 `std::sort` 是不稳定排序，相同熟悉度的卡片相对顺序可能改变，但这不影响功能。

进度保存：采用 `QMap<int, T>` 以卡组索引为键，实现 $O(\log n)$ 的查找和插入。由于索引可能因删除卡组而发生变化，实际使用中需要注意：删除卡组后，后续卡组的索引会减 1，导致 `QMap` 中保存的索引失效。为解决此问题，可以考虑改用卡组名称作为键，或每次删除后重建映射。当前实现假定用户不会频繁删除卡组，且索引变化后旧进度会被覆盖，属于已知的局限性。

JSON 序列化：利用 Qt 内置的 `QJsonDocument`、`QJsonObject`、`QJsonArray` 类，递归地将 `Deck` 和 `Card` 对象转换为 JSON 结构。输出格式为缩进文本，便于人工阅读和调试。

3. 小组成员分工情况

成员	职责	贡献内容
刘禹贤（组长）	项目架构设计、核心模块开发、代码整合与测试	设计 <code>DataManager</code> 单例模式、 <code>Card / Deck</code> 类；实现数据序列化与持久化；编写 <code>MainWindow</code> 的UI搭建与大部分业务逻辑；协调组内任务；撰写报告主体。
钟君浩	记忆复习与自测模块开发	实现记忆复习页面的循环浏览、进度保存/恢复；实现自测页面的评分系统、熟悉度排序算法、“再来一次”与“清除缓存”功能；协助调试进度保存相关的Bug。
孟宇翔	界面美化与用户体验优化	设计启动页、菜单页的视觉风格；实现背景更换功能；优化按钮状态联动（禁用/启用逻辑）；修复多处交互Bug；编写用户使用说明。

合作方式：三人通过 Git 进行版本管理，使用 GitHub 仓库托管代码。每周召开两次线上会议讨论需求变更和技术难题。刘禹贤负责最终代码审查与集成，确保代码风格一致且无编译错误。

4. 项目总结与反思

4.1 项目成果

成功开发了一款功能完整的复习卡片应用，支持卡组管理、两种复习模式、评分反馈和进度续接。

代码结构清晰，遵循面向对象原则，易于扩展（例如可添加更多复习策略）。数据持久化可靠，用户数据安全存储在本地 JSON 文件中。

界面美观，交互流畅，按钮状态联动合理，用户体验良好。

4.2 技术难点与解决方案

进度保存与恢复：最初采用全局变量记录进度，但切换卡组时容易混乱。后来改用 `QMap<int, ...>` 按卡组索引存储，并在切换时自动保存/加载，解决了数据冲突。

评分数据同步：自测时复制的卡片列表与原卡组数据独立，评分后需同步回原卡组。通过遍历原卡组匹配问题内容和答案内容实现，虽然效率不高但数据量小可接受。未来可改进为在开始自测时记录原始索引映射，避免遍历匹配。

UI 状态管理：多个按钮的启用/禁用状态复杂，容易遗漏。通过在每个槽函数中显式设置所有相关按钮状态，并提取公共重置函数，提高了代码可维护性。

索引失效问题：删除卡组后，后续卡组的索引发生变化，导致 `QMap` 中保存的进度指向错误卡组。当前采用折衷方案：删除卡组时清除该卡组及所有后续卡组的进度，并提示用户。更好的方案是改用卡组名称作为键，但名称可能重复，需引入 UUID。

4.3 不足与改进方向

性能：当卡组卡片数量极大（如数千张）时，排序和 JSON 读写可能稍慢。可考虑引入数据库（SQLite）替代 JSON 文件，并利用索引加速查询。

功能缺失：缺乏卡片导入/导出功能（如 CSV），也不支持图片或富文本。后续可添加多媒体支持，并实现 Anki 格式兼容。

跨平台适配：当前仅测试 Windows 环境，字体和布局在 macOS/Linux 上可能需要微调。计划使用 Qt 样式表统一外观，并在不同平台上进行测试。

单元测试：未编写自动化测试，手工测试覆盖率有限。未来可使用 Qt Test 框架编写单元测试，特别是针对 `Card::getFamiliarity()`、`Deck::getDueCards()` 等关键方法。

安全性：数据文件明文存储，未加密。若用户有隐私需求，可考虑使用简单的对称加密。

4.4 心得体会

通过本次大作业，小组成员深入掌握了 Qt 框架的 UI 编程、信号槽机制、文件 I/O 以及设计模式（单例）的应用。团队协作中锻炼了沟通与分工能力，认识到清晰的需求分析和合理的架构设计对项目成功的重要性。虽然过程中遇到诸多 Bug，但通过反复调试和代码重构，最终交付了满意的作品。这次实践为后续学习更复杂的软件开发打下了坚实基础。